Pad Controller / Drum Machine / Beatmaker

Arquitectura recomendada, hardware, firmware y plan de ejecución (4 meses / 5 personas)

Resumen del enfoque elegido
1. Arquitectura de hardware
2. Firmware y procesamiento de audio
3. Referencia de firmware real: Dirtywave M8
4. Plan de ejecución (4 meses / 5 personas)

Resumen del enfoque elegido

De las arquitecturas posibles para este tipo de producto, se recomienda construir un **groovebox autónomo con salida MIDI USB**: el dispositivo genera y reproduce su propio audio (no depende de un DAW ni de una computadora para sonar), incluye un secuenciador de patrones tipo step-sequencer, y adicionalmente puede funcionar como controlador MIDI de un DAW cuando se conecta por USB.

Se descarta explícitamente para esta primera versión la captura de audio en vivo (looper con micrófono/entrada de línea), porque es la funcionalidad de mayor riesgo técnico (sincronización de capas, aislamiento de ruido analógico) y la que menos aporta frente al tiempo disponible.

Plataforma de hardware: Teensy 4.1 (ARM Cortex-M7 a 600 MHz) + Teensy Audio Library.

---

1. Arquitectura de hardware

1.1 Microcontrolador

Teensy 4.1, basado en el procesador NXP MIMXRT1062 (ARM Cortex-M7 a 600 MHz):

- 1984 KB de Flash y 1024 KB de RAM (512 KB fuertemente acoplada, de acceso muy rápido para el pipeline de audio).
- 2 puertos I2S/TDM y 1 puerto S/PDIF de audio digital nativos — permiten conectar el códec de audio sin lógica adicional de conversión.
- 32 canales DMA de propósito general, esenciales para mover datos de audio sin bloquear al procesador.
- Procesador superescalar de doble emisión, capaz de ejecutar dos instrucciones por ciclo de reloj — margen de cómputo suficiente para varias voces de sample simultáneas más el secuenciador y la UI.
- Socket de microSD nativo, ideal para almacenar el banco de samples/kits.

Motivo de la elección: de todas las plataformas evaluadas (ESP32-S3, STM32, Raspberry Pi CM4/Linux), es la que ofrece el menor riesgo de ejecución en 4 meses: cuenta con una librería de audio

madura (Teensy Audio Library) con bloques ya probados para mezcla, reproducción de SD y grabación, evitando que el equipo tenga que escribir un pipeline DSP desde cero.

1.2 Sensado de los pads (16 pads, matriz 4×4)

Tecnología: sensores FSR (Force Sensing Resistor).

- Ofrecen una curva de *velocity* continua y natural (similar a MPC/Elektron), y permiten añadir *aftertouch* en el futuro con el mismo hardware, leyendo el sensor de forma continua en lugar de solo en el pico del golpe.
- **Multiplexado:** un multiplexor analógico 74HC4067 (16 canales) reduce las 16 señales analógicas de los FSR a 1 línea + 4 bits de selección, liberando pines del MCU.
- **Acondicionamiento de señal:** filtro paso-bajo RC (~1-2 kHz de corte) en cada canal antes del multiplexado, para atenuar ruido de alta frecuencia.
- **Prevención de crosstalk:** ventana de debounce en firmware que ignora picos detectados en un pad si llegan menos de 2-3 ms después de un golpe ya registrado en un pad adyacente.
- **Cálculo de velocity curve:** el valor crudo leído en una ventana de ~3-5 ms tras cruzar el umbral se mapea a 0-127 mediante una tabla de lookup no lineal (curva logarítmica/exponencial ajustable), reflejando mejor la percepción humana de “fuerza de golpe” que un mapeo lineal.

1.3 Controles adicionales

- **Encoders rotativos** para navegación de menús/parámetros, leídos por interrupción de cuadratura para no perder pasos en giros rápidos.
- **1 pantalla OLED (SSD1306, I2C)** para mostrar BPM, patrón activo y kit seleccionado — bajo consumo y muy simple de integrar.
- **LEDs RGB direccionables (WS2812B/SK6812)** en cada pad para dar feedback visual del secuenciador (paso activo, pad presionado). Deben controlarse por un periférico dedicado (DMA/temporizador propio) para que su temporización estricta no interfiera con el bus I2S de audio.

1.4 Subsistema de audio

- **Códec recomendado: PCM5102** (DAC estéreo I2S, sin registros de configuración I2C, “plug and play”) — simplifica el bring-up al eliminar una capa de configuración por software.
- **Aislamiento de ruido:** separar los planos de tierra analógica y digital, uniéndolos en un único punto (*star ground*) cerca del códec, y alimentar la sección analógica con un regulador LDO de bajo ruido dedicado, distinto del riel digital de 3.3V del MCU — evita el “whining” causado por la conmutación de líneas PWM/SPI de alta frecuencia acoplándose en la referencia de audio.

---

2. Firmware y procesamiento de audio

2.1 Motor de sonido

- **Reproductor de samples polifónico** (8-16 voces simultáneas) desde microSD, usando los bloques ya construidos de la **Teensy Audio Library** (mezclador, reproductor de SD, control de ganancia) — minimiza el código DSP escrito a mano.
- Sin síntesis compleja en tiempo real en esta versión (queda como

posible fase 2 sin cambios de hardware).
- **Gestión de memoria:** el kit de samples "activo" se carga en RAM al seleccionarlo; la lectura de nuevos samples desde SD se hace con *prefetch* anticipado para evitar cortes audibles por latencia variable de la tarjeta.

2.2 Secuenciador

- Step-sequencer de 16 pasos con múltiples patrones encadenables — es la funcionalidad que más valor aporta como producto "beatmaker" y es alcanzable con recursos moderados.
- Temporización referenciada a un contador de muestras de audio (no a un timer de "tiempo real"), para evitar deriva acumulada entre patrones.

2.3 MIDI

- **USB-MIDI Class Compliant** vía la librería nativa `usbMIDI` de Teensy — el dispositivo se reconoce automáticamente como controlador MIDI en cualquier DAW sin drivers adicionales.
- MIDI DIN clásico (UART a 31.25 kbaud, con opto-aislador en la entrada) como conector físico opcional, sujeto a tiempo disponible.

2.4 Prioridades de tiempo real (firmware bare-metal)

Para mantener la latencia de golpe-a-sonido por debajo de 10 ms:

1. **Máxima prioridad:** interrupción/DMA del códec de audio (nunca debe bloquearse).
2. **Alta:** escaneo de pads, en un temporizador periódico corto (~1 ms).
3. **Media:** generación y envío de mensajes MIDI.
4. **Baja:** refresco de pantalla OLED y animación de LEDs — primeras en saltarse un ciclo si el sistema está sobrecargado, porque no afectan el sonido.

---

3. Referencia de firmware real: Dirtywave M8

El Dirtywave M8 es la referencia de código abierto más directamente aplicable a este proyecto: es un tracker/groovebox portátil basado también en un microcontrolador Teensy, con ocho pistas, síntesis de forma de onda/FM/análoga virtual y reproducción de samples desde microSD, con su firmware publicado públicamente en GitHub (`Dirtywave/M8Firmware` y `M8HeadlessFirmware`). Se recomienda que el equipo de firmware revise su arquitectura de pipeline de audio como plantilla real de cómo estructurar el motor de sonido y el secuenciador sobre la misma familia de MCU que se usará en este proyecto.

---

4. Plan de ejecución (4 meses / 5 personas)

4.1 Roles

Rol	Responsabilidades principales

1. Hardware / PCB	Esquemático, selección de componentes, diseño de PCB (multiplexor de pads, códec, alimentación), fabricación (Gerbers + BOM), pruebas eléctricas
2. Firmware — Audio/DSP	Pipeline de audio, reproductor de samples, mezclador, gestión de SD, optimización de latencia
3. Firmware — Control/MIDI	Escaneo de pads y velocity curve, encoders, secuenciador, stack USB-MIDI, UI en OLED
4. Integración mecánica / CAD 3D	Diseño de carcasa, disposición ergonómica, ensamblaje, cableado interno
5. Pruebas / Software host / Documentación	Plan de pruebas (latencia, crosstalk, consumo), herramienta host para cargar kits, documentación de usuario, coordinación del cronograma

4.2 Cronograma (semanas 1-16)

Semanas	Fase	Entregable clave
1-2	Especificación congelada, pedido de componentes de evaluación	Documento de especificación (evitar *scope creep*)
3-5	Prototipado en protoboard: 1-2 pads con FSR + multiplexor, audio básico desde SD, primer mensaje MIDI USB	Subsistemas funcionando por separado
6-7	Integración de los 16 pads + códec + primer secuenciador	Prototipo funcional en mesa de laboratorio
8-9	Diseño de PCB v1 y envío a fabricación; diseño de carcasa CAD en paralelo	Gerbers enviados; modelo 3D de carcasa
10-11	*Bring-up* de PCB v1 (verificación eléctrica y corrección de errores)	PCB v1 validada
12-13	Firmware final: UI completa, velocity curve calibrada, kits definitivos, pruebas de estrés	Firmware “feature-complete”
14	Ensamblaje en carcasa definitiva	Unidad(es) terminadas
15	Pruebas de usuario con músicos externos, ajustes de firmware (no de hardware)	Reporte de pruebas
16	Pulido final, documentación,	Producto y documentación

Nota: el cronograma asume una sola iteración de PCB. Un error de diseño no corregible por software en la semana 10-11 obligaría a una revisión v2, con 2-3 semanas adicionales.

4.3 Riesgos técnicos principales y mitigación

1. **Crosstalk entre pads / velocity poco confiable** → validar el circuito de acondicionamiento sobre un solo pad en protoboard desde la semana 3, antes de integrar los 16.
2. **Ruido digital acoplándose al audio (whining, clics)** → definir la separación de planos de tierra y la alimentación analógica dedicada desde el primer esquemático, no como arreglo posterior.
3. **Subestimación del tiempo de *bring-up* firmware-hardware** → reservar 2 semanas completas (10-11) exclusivamente para esto, sin desarrollo de features nuevas en paralelo.